

# Optimal Ship Routing

## Using Pathfinding Algorithms, Geospatial Constraints & Machine Learning

*Detailed End-to-End Project Report*

| Student           | USN        |
|-------------------|------------|
| K. Aravind Kamath | NNM23IS082 |
| Manisha G. P.     | NNM23IS093 |
| Drupad            | NNM23IS064 |
| Dhanvee           | NNM23IS055 |

**Project Code:** 43 | **Guide:** Mr. Prashanth Kumar  
Department of Information Science & Engineering

# 1 Abstract

This project builds an intelligent system that computes the best sea route for a ship travelling between Indian ports. Instead of blindly following the shortest straight line, it reasons about the real ocean: how deep the water is, how close the coast is, and how rough the waves are for the specific vessel being sailed. It combines three ideas: (1) **classical pathfinding** (Dijkstra and a weather-aware A\*), (2) **geospatial constraints** from real bathymetry and coastline data, and (3) a **machine-learning speed model** that predicts how much waves slow a given ship. The result is a route that is safer, calmer and more fuel-efficient. Everything is free, runs offline on an ordinary laptop, and is presented through an interactive web app ('Nautilus') built on Google Maps.

## 2 Why This Project?

### 2.1 The problem with traditional routing

Traditional routing relies on static charts, fixed historical routes and rule-based navigation. These work in calm, predictable conditions but fail to adapt to changing weather and sea state. The consequences are real: more fuel burned in heavy seas, higher operating cost, more CO<sub>2</sub> emissions, and safety risk when routes pass too close to shallow water or coastlines.

### 2.2 Why not the heavy research alternatives

Some modern systems use deep reinforcement learning or large optimisation solvers. They are powerful but heavy: GPUs, big datasets, long training, and decisions that are hard to explain in a viva. Our goal was a system that is **fast, explainable, adaptable and completely free** — which pathfinding plus a light ML model delivers.

## 3 System Overview (End-to-End)

The system is a pipeline; each stage produces a small cached output so the next stage is fast and the laptop is never overloaded.

| Stage          | What happens                                                     | Output                           |
|----------------|------------------------------------------------------------------|----------------------------------|
| 1. Preprocess  | Huge raw datasets sliced to the region, downsampled, saved.      | bathymetry.npz, weather.npz, CSV |
| 2. Grid model  | Ocean turned into safe/unsafe cells; KD-tree for snapping ports. | grid.npz + KD-tree               |
| 3. ML model    | Speed predictor trained on wave-speed data.                      | speed_model.joblib               |
| 4. Weather     | Waves mapped to the grid (typical / rough / live).               | Weather field                    |
| 5. Routing     | Dijkstra and weather-aware A* find the route.                    | Route + statistics               |
| 6. Evaluate    | Proposed vs traditional on 4 metrics.                            | evaluation.csv                   |
| 7. Nautilus UI | Google-Maps web app shows the result.                            | Web dashboard                    |

## 4 The Datasets — What & Why

### 4.1 GEBCO 2024 bathymetry (sea-floor depth)

**What:** a global map of ocean depth in metres at a fine 15 arc-second grid (43,200×86,400 points). Negative values are depth; values  $\geq 0$  are land. **Why:** depth is the most important safety input — it tells us where it is deep enough to sail, where land is, and where shallow shoals hide. It is authoritative, global and free.

#### Why not the alternatives:

- *Electronic Nautical Charts (ENC/S-57)*: excellent but often licensed/paid and region-fragmented — breaks the free rule.
- *Coarse models (ETOPO 1 arc-min)*: free but too low-resolution, so narrow safe channels get lost.
- *GEBCO* gives the best balance: free, global, fine enough to resolve coastlines and shoals.

### 4.2 ERA5 wave data (the weather)

**What:** an ECMWF ERA5 file with three wave variables on a 0.5° grid, hourly, for ~7 months (5,088 time steps): significant wave height (**swh**), mean wave direction (**mwd**) and mean wave period (**mwp**). **Why these three:** they determine how hard a passage is. Height is the main danger and fuel penalty; direction tells us if the ship sails *into* the waves (head seas, worst case) or with them; period separates long gentle swell from short choppy seas.

#### Why not the alternatives:

- *Wind speed alone*: too indirect — the resulting waves are what actually slow and endanger the ship.
- *Ocean currents*: a smaller effect than waves for surface vessels; extra complexity for little gain at this scale.
- *ERA5* is free, trusted and complete for height, direction and period together.

### 4.3 Open-Meteo Marine API (live weather)

**What:** a free web service returning the *current* wave height, direction and period for any coordinate. **Why this one:** it is genuinely free — **no API key and no credit card** — unlike commercial marine-weather APIs, so it keeps the zero-cost promise while adding a real-time capability.

### 4.4 Dataset summary

| Dataset    | Role                        | Resolution   | Cost          |
|------------|-----------------------------|--------------|---------------|
| GEBCO 2024 | Depth / land / shoals       | 15 arc-sec   | Free          |
| ERA5 waves | Stored weather (typ./rough) | 0.5°, hourly | Free          |
| Open-Meteo | Live weather                | On-demand    | Free (no key) |

## 5 Every Feature, Explained in Detail

---

### 5.1 One-time data ‘shrink’ (preprocessing)

The raw files are enormous (GEBCO 7.4 GB, waves 4.1 GB). Loading them whole would freeze a laptop. The preprocessor opens them **lazily** (xarray + dask stream the data in small blocks), cuts out only the seas around India, downsamples and writes tiny files.

- **Why lazy/dask:** memory stays low — the full array is never in RAM at once. This is the key reason the laptop never hangs.
- **Why keep the shallowest point when downsampling:** when 12 fine cells become one, we keep the most-shallow value, not the average — averaging could hide a dangerous shoal.
- Two weather scenarios are precomputed (*typical* = time-average, *rough* = 95th-percentile storm) plus a normalised CSV.

**Result:** 11.6 GB of raw data becomes under 1.1 MB of working data; the big files can then be deleted.

### 5.2 Grid model (a navigable ocean map)

The region (lat 0–27°N, lon 50–100°E) is modelled as a grid at 0.05° spacing (~5.5 km): 540×1000 = 540,000 cells, of which **337,196 (62%) are navigable**. A cell is navigable only if the water is deep enough (below –10 m) and it is at least **22 km from the coast** (safety buffer). The buffer is computed with a **distance transform**; a **KD-tree** then snaps any port to the nearest safe cell, restricted to the main connected ocean body so a port can never land in an unreachable pocket.

#### Why a grid (not a road-style or visibility graph)?

- The open ocean has no roads — a uniform grid means ‘go anywhere that is safe’.
- It maps one-to-one onto the raster depth/weather data.
- It makes attaching a weather cost to every possible move trivial.

### 5.3 Weather layer & cost function

The coarse wave grid is interpolated onto the fine routing grid. The heart of the weather-aware behaviour is a **cost multiplier** for every move: higher waves cost more; sailing into the waves (head seas) costs more still; short choppy periods cost slightly more. It is a simple, **explainable** formula — you can point to exactly why the route bends. Three scenarios exist: **Typical**, **Rough** and **Live**.

### 5.4 Vessel parameters

Five vessel types are modelled (Bulk Carrier, Container Ship, VLCC Tanker, General Cargo, LNG Carrier), each with service speed, length, draft and a **wave-sensitivity** factor. This matters because a large container ship powers through waves that would force a small cargo ship to detour — so the *same* sea state gives *different* optimal routes for different ships.

### 5.5 Machine-learning speed model

**What it does:** a scikit-learn **Random Forest** predicts the *attainable speed* of the chosen vessel given the local sea state (wave height, relative wave angle, period) and the vessel parameters. The weather-aware A\* then uses **cost = distance ÷ predicted speed** — real travel time — so routing directly optimises for the conditions this ship actually faces. This is the ‘ML-based decision technique’ named in the synopsis.

#### Why Random Forest (not deep learning / RL)?

- **Trains in ~2 seconds on a CPU** — no GPU, no cost, no laptop strain. Deep RL would need hours and hardware.
- **Accurate and stable** here (test  $R^2 = 0.998$ , error 0.17 knots).
- **Robust and explainable** — easy to defend in a viva.

**Why a synthesised training set?** Real per-ship speed-loss logs are not public, so we generate data from an established added-resistance relation (wave resistance grows ~ with wave height squared, worst in head seas), add noise, and let the model learn it. It can later be retrained on real AIS data with no code change.

## 5.6 Routing engine — Dijkstra & weather-aware A\*

Both search the grid with 8-directional moves (including diagonals). **Dijkstra** ignores weather and finds the shortest *safe* distance — the traditional baseline. **Weather-aware A\*** uses the ML/weather cost plus a great-circle **heuristic** that guides the search toward the goal — the proposed system.

### Why A\*, and why both together?

- A\*'s heuristic explores far fewer cells than Dijkstra (about **74% fewer** in tests) while still returning the optimal route — the heuristic is *admissible*, which guarantees optimality.
- Dijkstra as a baseline lets us **prove** the weather-aware route is better, exactly as the synopsis asks.
- Per-move distances and weather multipliers are **pre-computed**, so a full route solves in ~1–2 seconds.

## 5.7 Evaluation module (proposed vs traditional)

A\* is compared with Dijkstra across several voyages on the four metrics the synopsis names:

| Metric                   | How it is measured                                |
|--------------------------|---------------------------------------------------|
| Computational efficiency | Cells explored + runtime (A* ~74% fewer).         |
| Route optimality         | Total distance and estimated travel time.         |
| Safety compliance        | Min distance-to-coast on route ( $\geq 22$ km).   |
| Adaptability             | Peak/mean wave height experienced (A* lowers it). |

Wave exposure is measured along *both* routes using the *same* weather field, so the comparison is honest.

## 5.8 The Nautilus web application

The main front-end is a polished dashboard on **Google Maps** (satellite/road, smooth animated route, live wave overlay, ship & anchor endpoint icons, clickable weather tooltips, side-by-side metric cards) served by a small **Flask** backend. A key-free **Streamlit** app with free open-source maps is included as a fallback.

### Two engineering points worth mentioning:

- Google removed the old *HeatmapLayer*, so the wave overlay is rendered **server-side as a PNG** and placed as a ground overlay — smoother and future-proof.
- The Google Maps key lives in a **private, git-ignored file** and is restricted by website + API, so it is never exposed when sharing.

## 6 Key Design Decisions: Why X instead of Y

| Decision      | Chosen               | Why not the alternative                                            |
|---------------|----------------------|--------------------------------------------------------------------|
| Depth data    | GEBCO 2024           | Charts are paid/licensed; ETOPO too coarse.                        |
| Weather       | ERA5 waves           | Wind-only is indirect; currents add cost for little gain.          |
| Live weather  | Open-Meteo           | Commercial marine APIs need a paid key / card.                     |
| Ocean model   | Uniform grid         | Road/visibility graphs don't suit open sea.                        |
| Search        | Dijkstra + A*        | Deep RL / genetic solvers need GPUs and long training.             |
| ML model      | Random Forest        | Deep learning needs GPU/hours; RF trains in seconds, $R^2=0.998$ . |
| Big files     | Lazy (dask)          | Loading whole files hangs the laptop.                              |
| Downsampling  | Keep shallowest      | Averaging could erase a shoal.                                     |
| Port snapping | KD-tree + main basin | Guarantees a reachable safe cell.                                  |
| Main UI map   | Google Maps          | User's own key; Streamlit map kept as fallback.                    |
| Wave overlay  | Server PNG overlay   | Google deprecated the JS HeatmapLayer.                             |

## 7 Constraints & How They Are Met

| Constraint     | How it is satisfied                                                       |
|----------------|---------------------------------------------------------------------------|
| No laptop hang | Lazy/streamed loading; pre-computed costs; routing in 1–2 s.              |
| Low storage    | 11.6 GB raw → ~1 MB working data (+11 MB model); raw files deletable.     |
| Zero cost      | All open-source; free datasets; free Open-Meteo; no cloud / subscription. |

## 8 Representative Results

Rough scenario, General Cargo / VLCC (from evaluation.csv):

- **Adaptability / safety:** Cochin→Dubai, A\* trades about **+298 km** to cut the peak wave from **4.13 m** to **3.31 m**.
- **Computational efficiency:** A\* explored **~74% fewer cells** than Dijkstra on average.
- **Safety compliance:** every route stayed  $\geq 22$  km from the coast.
- **Live capability:** using real-time Open-Meteo data, the system routed around today's actual rough patches.

## 9 Technology Stack

| Layer    | Tools (all free / open-source)       |
|----------|--------------------------------------|
| Language | Python 3                             |
| Data     | xarray, dask, netCDF4, numpy, pandas |

| Layer                   | Tools (all free / open-source)                               |
|-------------------------|--------------------------------------------------------------|
| Geospatial / algorithms | scipy (KD-tree, distance transform), custom Dijkstra & A*    |
| Machine learning        | scikit-learn (Random Forest), joblib                         |
| Visualisation           | matplotlib (wave PNG)                                        |
| Web                     | Flask + Google Maps JS (main); Streamlit + Folium (fallback) |
| Live weather            | Open-Meteo Marine API (requests)                             |

## 10 Limitations & Future Scope

---

- Results depend on the accuracy/coverage of the depth and weather data.
- The region is the seas around India; the bounding box can be widened for global voyages by re-running preprocessing.
- The ML model uses a physics-based synthetic dataset; it can be retrained on **real AIS speed logs** for site-specific accuracy.
- Live weather needs internet; offline Typical/Rough cover demos without connectivity.
- Future: ocean currents, traffic-density avoidance, and a forecast time-slider.

## 11 Conclusion

---

The system delivers exactly what the synopsis set out: an intelligent, adaptive ship-routing tool combining bathymetry, weather and machine-learning decision-making with classical pathfinding. It produces safer, calmer, more efficient routes, proves its advantage against a traditional baseline, and presents everything through an interactive Nautilus web interface — all while running for free, offline, and without straining an ordinary laptop.